

Your connections

- LSM303DLHC VCC → Pico Pin 36 (3.3V OUT)
- LSM303DLHC GND → Pico Pin 38 (GND)
- LSM303DLHC SDA → GP0
- LSM303DLHC SCL → GP1
- I2C Bus → I2C(0)

The detected addresses are typically:

- Accelerometer → 0x19
- Magnetometer → 0x1E

First, an important point about distance

The LSM303DLHC cannot accurately calculate distance.

It only contains:

- 3-axis Accelerometer
- 3-axis Magnetometer

It does not contain a gyroscope, GPS, or optical tracking.

Distance is obtained by

Acceleration



Velocity



Distance

This requires double integration, and even a tiny sensor error becomes huge after a few seconds.

Example

Actual acceleration error = 0.03 m/s^2

After 10 seconds

Velocity error $\approx 0.3 \text{ m/s}$

Distance error $\approx 1.5 \text{ meters}$

After 30 seconds the error becomes tens of meters.

So no professional system calculates distance using only an LSM303DLHC.

Professional systems combine

- IMU

- GPS
 - Wheel encoder
 - Camera
 - LiDAR
 - Kalman Filter
-

Real applications of LSM303DLHC

1. Digital Compass

Uses only the magnetometer.

Applications

- Mobile phones
- Marine navigation
- Hiking devices
- Robot heading

Output

Heading = 0° → North

90° → East

180° → South

270° → West

2. Tilt Meter

Uses accelerometer.

Applications

- Camera gimbal
- Crane leveling
- Construction
- Solar panel alignment

Output

Roll = 10°

Pitch = -5°

3. Fall Detection

Applications

- Smart watch
- Elderly monitoring
- Accident detection

Detects

Free fall

Impact

No movement

4. Motion Detection

Applications

- Burglar alarm
 - Vehicle anti-theft
 - Asset tracking
-

5. Step Counter

Applications

- Fitness bands
 - Health monitoring
-

6. Robot Balancing

Applications

- Self balancing robot

Uses

Roll

Pitch

7. Drone Orientation

Applications

- **Drone stabilization**

Uses

Accelerometer

Magnetometer

8. Gesture Detection

Applications

- **Smart remote**
 - **Air mouse**
-

9. Vehicle Inclination

Applications

- **Truck**
 - **Excavator**
 - **Mining**
-

10. Earth Magnetic Field Measurement

Applications

- **Geological survey**
 - **Metal detection**
-

Can it estimate distance?

Yes, only for a few centimeters if the motion lasts less than about 1 second and the sensor is well calibrated.

Example

Move sensor

20 cm

Estimate

18–24 cm

After a few seconds

Real = 2 m

Estimated = 7 m

Complete pipeline

Read raw bytes

↓

Convert to signed integers

↓

Scale to g

↓

Remove bias

↓

Low-pass filter

↓

Convert to m/s^2

↓

Subtract gravity

↓

Velocity integration

↓

Distance integration

Programs you should write (recommended order)

Program 1

I2C Scanner

Output

Found device

0x19

0x1E

Program 2

Read Accelerometer Raw Data

Output

X

Y

Z

Program 3

Convert Raw to Signed

Output

X = -120

Y = 520

Z = 16020

Program 4

Convert into g

Output

X = 0.03g

Y = -0.01g

Z = 0.98g

Program 5

Bias Calibration

Keep sensor still

Collect

500 samples

Average

Subtract bias

Program 6

Remove Gravity

If flat

Z

≈1g

↓

0g

Program 7

Calculate Roll

Roll =

atan2(Y,Z)

Program 8

Calculate Pitch

Pitch =

**atan2(-X,
sqrt(Y²+Z²))**

Program 9

Read Magnetometer

Calculate Heading

Heading

0–360°

Program 10

Velocity

$$v=v+a \times dt$$

Program 11

Distance

$$d=d+v \times dt$$

Only reliable for very short durations due to drift.

Avoiding EIO and OSError

These errors usually happen because:

- Wrong device address.
- Reading registers before the sensor is initialized.
- I2C clock too fast for wiring quality.
- Missing pull-up resistors (many breakout boards already include them).
- Reading the wrong number of bytes or from invalid registers.
- Loose or noisy jumper wires.

To minimize them:

- Use `I2C(0, scl=Pin(1), sda=Pin(0), freq=100000)` (100 kHz is more robust than 400 kHz).
 - Verify the device address is present before each read.
 - Initialize the accelerometer control registers before reading data.
 - Wrap I2C reads in `try/except OSError` and retry a few times rather than crashing.
 - Confirm your module is powered from 3.3 V, not 5 V.
-

Recommended project progression

1. I2C Scanner ✓ (already done)
2. Accelerometer Raw Data
3. Magnetometer Raw Data
4. Signed Integer Conversion
5. Accelerometer Calibration (bias removal)
6. Roll & Pitch Calculation
7. Compass (Heading)
8. Motion Detection

9. Step Counter

10. Short-duration Velocity Estimation

11. Short-duration Distance Estimation (demonstration only)

This sequence builds from reliable sensor communication to increasingly advanced processing, and it avoids many common I2C issues. For a Raspberry Pi Pico running MicroPython in Thonny, each stage should be developed and tested independently before moving to the next.

ChatGPT said:

Excellent. This is where your project starts behaving like a real embedded defense application.

Until now, we have:

- ☒ I2C Communication
- ☒ Register Initialization
- ☒ Raw Data Reading
- ☒ Two's Complement Conversion
- ☒ Bias Calibration
- ☒ Roll & Pitch Calculation

But if you run the program now, you'll notice that the values still fluctuate slightly even when the sensor is perfectly still.

For example:

Roll = 0.62°

Roll = -0.41°

Roll = 1.15°

Roll = -0.75°

Nothing is moving, yet the values change. This is called sensor noise.

Step 7 - Low Pass Filtering (Noise Removal)

Why do we need a filter?

The accelerometer measures extremely small changes in acceleration. Electrical noise, ADC quantization, and tiny vibrations from the table or your hand cause the readings to fluctuate.

Without filtering:

Actual Tilt = 0°

Measured

0.5°
-0.7°
1.1°
-0.2°
0.8°

With filtering:

Actual Tilt = 0°

Measured

0.03°
0.05°
0.02°
0.01°

This makes the system much more stable.

Real Defence Example

Imagine an ammunition box stored in an armoury.

Without filtering:

Tiny vibration

↓

Roll changes by 2°

↓

False Alarm

With filtering:

Tiny vibration

↓

Ignored

↓

No Alarm

Types of Filters

There are many filters:

- Moving Average Filter
- Low Pass Filter
- Complementary Filter
- Kalman Filter
- Butterworth Filter

For the LSM303DLHC on a Raspberry Pi Pico, we'll use a First Order Low Pass Filter because it is:

- Simple
 - Fast
 - Low memory
 - Common in embedded systems
-

Formula

The low-pass filter uses this equation:

$$Filtered = \alpha \times New + (1 - \alpha) \times Previous$$

where:

- New = current sensor reading
 - Previous = previous filtered value
 - α (alpha) = smoothing factor (0 to 1)
-

Understanding Alpha

If:

Alpha = 1.0

No filtering.

If:

Alpha = 0.5

Medium filtering.

If:

Alpha = 0.2

Strong filtering.

We'll use:

Python

Run

ALPHA = 0.20

This is a good starting point.

Global Variables

Add these near the top of your program:

Python

Run

```
# -----  
# Low Pass Filter Variables  
# -----
```

ALPHA = 0.20

filtered_roll = 0.0

filtered_pitch = 0.0

Filter Function

Add this function:

Python

Run

```
# -----  
# Low Pass Filter  
# -----
```

def low_pass_filter(roll, pitch):

```
    global filtered_roll  
    global filtered_pitch
```

```
    filtered_roll = (  
        ALPHA * roll  
        +  
        (1 - ALPHA) * filtered_roll  
    )
```

```
    filtered_pitch = (
```

```
    ALPHA * pitch
    +
    (1 - ALPHA) * filtered_pitch
)

return filtered_roll, filtered_pitch
```

Line-by-Line Explanation

Function Definition

Python

Run

```
def low_pass_filter(roll, pitch):
```

The function receives the current roll and pitch.

Global Variables

Python

Run

```
global filtered_roll
```

The previous filtered value must be remembered for the next calculation.

Formula

Suppose:

Previous Roll = 8°

Current Roll = 12°

Alpha = 0.2

Then:

Filtered

=

0.2×12

+

0.8×8

=

2.4

+

6.4

=

8.8°

Instead of jumping directly to 12°, the filter moves gradually to 8.8°.

Return

Python

Run

```
return filtered_roll, filtered_pitch
```

These are the values that should be used in the rest of the program.

Update Main Loop

Replace:

Python

Run

```
roll, pitch = calculate_angles(x, y, z)
```

with:

Python

Run

```
roll, pitch = calculate_angles(x, y, z)
```

```
roll, pitch = low_pass_filter(  
    roll,  
    pitch  
)
```

Updated Main Program

Python

Run

```
calibrate_accelerometer()
```

```
while True:
```

```
    values = get_calibrated_data()
```

```
    if values is not None:
```

```
        x, y, z = values
```

```
        roll, pitch = calculate_angles(x, y, z)
```

```
        roll, pitch = low_pass_filter(
            roll,
            pitch
        )
```

```
        print("-----")
```

```
        print("Roll =", round(roll,2))
```

```
        print("Pitch =", round(pitch,2))
```

```
    utime.sleep(0.1)
```

Expected Output

Sensor lying flat:

Roll = 0.05°

Pitch = -0.08°

Roll = 0.04°

Pitch = -0.06°

Roll = 0.02°

Pitch = -0.05°

Tilt the sensor slowly:

Roll

5°

10°

15°

20°

25°

The transition is smooth instead of jumping.

Why Not Use Raw Angles?

Suppose the raw data is:

10°

12°

9°

14°

11°

The filtered output becomes:

10°

10.4°

10.1°

10.9°

10.7°

This is much more stable and suitable for making decisions.

Defence Advantage

In a military equipment monitoring system:

Without filtering:

Vehicle engine vibration

↓

False Alarm

With filtering:

Vehicle vibration



Ignored



Alarm only for actual movement

Small Improvement to Our Code

Right now, `filtered_roll` and `filtered_pitch` start at 0.0. That means the first few outputs may take a moment to settle.

A common improvement is to initialize them with the first calculated roll and pitch instead of zero. This avoids the initial "ramp-up" effect.

For example:

Python

Run

```
filtered_roll = None
```

```
filtered_pitch = None
```

```
def low_pass_filter(roll, pitch):
```

```
    global filtered_roll, filtered_pitch
```

```
    if filtered_roll is None:
```

```
        filtered_roll = roll
```

```
        filtered_pitch = pitch
```

```
    return filtered_roll, filtered_pitch
```

```
    filtered_roll = ALPHA * roll + (1 - ALPHA) * filtered_roll
```

```
    filtered_pitch = ALPHA * pitch + (1 - ALPHA) * filtered_pitch
```

```
    return filtered_roll, filtered_pitch
```

This version is more typical of production embedded code.

Remaining Steps

Only 5 steps remain to complete the application:

1. Movement Detection (detect actual motion using acceleration changes)
2. Tilt Threshold Detection (detect excessive roll or pitch)

3. **Tamper Detection Decision Logic (combine movement and tilt to reduce false alarms)**
4. **Alarm Output (LED, buzzer, or console alert)**
5. **Final Integrated Program (all code combined into one clean, well-structured MicroPython application)**

At that point, you'll have a complete Military Equipment Tamper Detection system that can serve as the foundation for your other three applications.

You said:

next

ChatGPT said:

Excellent. Now we are going to build the decision-making part of the application.

Up to now, we've only been measuring the sensor. From this step onward, the Pico will begin interpreting the measurements to decide whether the equipment has actually been disturbed.

Step 8 – Movement Detection

Objective

Detect whether the equipment is moving by monitoring changes in acceleration.

Unlike tilt detection (which looks at orientation), movement detection looks for changes in acceleration over time.

Why Not Just Check X, Y, or Z?

Suppose the equipment is lying flat.

$X = 2$

$Y = -3$

$Z = 1022$

Now someone lifts it.

$X = 115$

$Y = 62$

Z = 965

Instead of checking each axis separately, it is more reliable to compute the overall change.

Method 1 (Recommended)

We'll calculate the difference between the current reading and the previous reading.

Previous Reading

↓

Current Reading

↓

Difference

↓

If Difference > Threshold

↓

Movement Detected

Variables Needed

Add these global variables:

Python

Run

```
# -----  
# Movement Detection Variables  
# -----
```

```
previous_x = None
```

```
previous_y = None
```

```
previous_z = None
```

```
MOVEMENT_THRESHOLD = 35
```

Why None?

When the program starts, there is no previous reading.

Using None lets us identify the very first sample.

Movement Detection Function

Add this function:

Python

Run

```
# -----  
# Detect Movement  
# -----  
  
def detect_movement(x, y, z):  
  
    global previous_x  
    global previous_y  
    global previous_z  
  
    # First sample  
    if previous_x is None:  
  
        previous_x = x  
        previous_y = y  
        previous_z = z  
  
        return False  
  
    dx = abs(x - previous_x)  
    dy = abs(y - previous_y)  
    dz = abs(z - previous_z)  
  
    previous_x = x  
    previous_y = y  
    previous_z = z  
  
    if (  
        dx > MOVEMENT_THRESHOLD or  
        dy > MOVEMENT_THRESHOLD or  
        dz > MOVEMENT_THRESHOLD  
    ):  
        return True  
  
    return False
```

Line-by-Line Explanation

First Sample

Python

Run

if previous_x is None:

This happens only once when the program starts.

We simply store the first reading.

Example:

Current

X = 5

Y = -3

Z = 1022

Store it.

No comparison is made.

Difference

Python

Run

dx = abs(x - previous_x)

Suppose

Previous

X = 25

Current

X = 60

Difference

60 - 25

=

35

Absolute value ensures the result is always positive.

Store Current Reading

Python

Run

`previous_x = x`

The current reading becomes the previous reading for the next loop.

Threshold

Suppose

`dx = 5`

`dy = 7`

`dz = 10`

Threshold

35

No movement.

Now

`dx = 58`

`dy = 20`

`dz = 11`

Movement detected.

Why Use a Threshold?

Sensors always have noise.

Without a threshold:

0.2

0.5

0.7

↓

Movement

False alarm.

With threshold:

0.2

0.5

0.7

↓

Ignored

Add to Main Loop

After

Python

Run

```
roll, pitch = low_pass_filter(  
    roll,  
    pitch  
)
```

add

Python

Run

```
movement = detect_movement(x, y, z)
```

```
print("Movement =", movement)
```

Expected Output

Sensor still

Movement = False

Shake sensor

Movement = True

Real Defence Example

Imagine an equipment box.

Someone touches it.

Acceleration changes

↓

Movement=True

If nobody touches it

Movement=False

Is This Enough?

Not yet.

Imagine this situation.

Someone slowly tilts the equipment.

Acceleration changes only a little.

Movement may remain

False

But

Roll = 28°

Clearly the equipment has been moved.

This is why professional systems combine multiple indicators rather than relying on a single measurement.

Step 9 – Tilt Detection (Next)

We'll use the filtered roll and pitch values.

For example:

If

$|Roll| > 15^\circ$

OR

$|Pitch| > 15^\circ$

↓

Equipment Tilted



Tilt = True

Then, in the final tamper logic, we can combine both conditions:

Movement OR Excessive Tilt



Tamper Detected



Alarm

This approach is more reliable than using only acceleration changes, because it can detect both sudden movement and slower changes in orientation.

You said:

next only give the major steps only including code not any simple steps

ChatGPT said:

Perfect. From now on, I'll skip the small theory and give you only the major implementation steps with complete code. After the final code, I'll explain the complete program.

For Military Equipment Tamper Detection, only these major steps remain:

1. Tilt Detection
2. Tamper Detection Logic
3. Alarm (LED/Buzzer)
4. Complete Integrated Program

Major Step 9: Tilt Detection

Python

Run

-----

Tilt Threshold

```
# -----  
  
TILT_THRESHOLD = 15    # Degrees  
  
def detect_tilt(roll, pitch):  
  
    if abs(roll) > TILT_THRESHOLD:  
        return True  
  
    if abs(pitch) > TILT_THRESHOLD:  
        return True  
  
    return False
```

Major Step 10: Tamper Detection Logic

Python

Run

```
# -----  
# Tamper Detection  
# -----  
  
def detect_tamper(movement, tilt):  
  
    if movement or tilt:  
        return True  
  
    return False
```

Major Step 11: Alarm

Connect

LED

GP15 -----220Ω-----LED-----GND

Code

Python

Run

```
from machine import Pin
```

```
led = Pin(15, Pin.OUT)
```

```
def alarm(status):
```

if status:

led.value(1)

```
print("*****")  
print(" TAMPER ALERT ")  
print("*****")
```

else:

led.value(0)

Major Step 12: Main Loop

Python

Run

calibrate_accelerometer()

while True:

values = get_calibrated_data()

if values is None:
continue

x, y, z = values

roll, pitch = calculate_angles(x, y, z)

```
roll, pitch = low_pass_filter(  
    roll,  
    pitch  
)
```

```
movement = detect_movement(  
    x,  
    y,  
    z  
)
```

```
tilt = detect_tilt(  
    roll,  
    pitch  
)
```

```

tamper = detect_tamper(
    movement,
    tilt
)

alarm(tamper)

print("-----")
print("X =", round(x,2))
print("Y =", round(y,2))
print("Z =", round(z,2))

print("Roll =", round(roll,2))
print("Pitch =", round(pitch,2))

print("Movement =", movement)
print("Tilt =", tilt)
print("Tamper =", tamper)

utime.sleep(0.1)

```

Final Program Structure

main.py

```

|

├— Imports

├— Register Definitions

├— I2C Initialization

├— Register Write

├— Register Read

├— Sensor Initialization

├— Read Accelerometer

├— Two's Complement Conversion

├— Calibration

├— Angle Calculation

```

└─ Low Pass Filter

└─ Movement Detection

└─ Tilt Detection

└─ Tamper Detection

└─ Alarm

└─ Main Loop

Project 1 Status

Module	Status
I2C Communication	✓
Sensor Initialization	✓
Register Reading	✓
Signed Conversion	✓
Calibration	✓
Roll Calculation	✓
Pitch Calculation	✓
Low Pass Filter	✓
Movement Detection	✓
Tilt Detection	✓

Module	Status
Tamper Detection	✓
LED Alarm	✓

One Important Recommendation Before You Use This

For a real deployment, I recommend changing the tamper decision from:

Python

Run

if movement or tilt:

to a slightly more robust version:

Python

Run

tamper = movement and tilt

or even:

Python

Run

tamper = movement and (abs(roll) > 15 or abs(pitch) > 15)

Why?

- movement OR tilt can generate false alarms if someone slowly tilts the device during maintenance or if vibration briefly exceeds the movement threshold.
- movement AND tilt requires both conditions to occur together, which is generally more reliable for a stored equipment monitoring application.

This simple change often reduces nuisance alarms without making the system much less sensitive.

complete code

```
from machine import Pin, I2C
```

```
import utime
```

```
import math
```

```
# =====
```

```
# CONFIGURATION
```

```
# =====
```

```
I2C_ID = 0
```

```
SDA_PIN = 0
```

```
SCL_PIN = 1
```

```
I2C_FREQ = 100000
```

```
ACC_ADDRESS = 0x19
```

```
CTRL_REG1_A = 0x20
```

```
CTRL_REG4_A = 0x23
```

```
OUT_X_L_A = 0x28
```

```
AUTO_INCREMENT = 0x80
```

```
LED_PIN = 15
```

```
MOVEMENT_THRESHOLD = 35
```

```
TILT_THRESHOLD = 15
```

```
ALPHA = 0.2
```

```
# =====
```

```
# GLOBAL VARIABLES
```

```
# =====
```

```
x_bias = 0
```

```
y_bias = 0
```

```
z_bias = 0
```

```
filtered_roll = None
```

```
filtered_pitch = None
```

```
previous_x = None
```

```
previous_y = None
```

```
previous_z = None
```

```
# =====
```

```
# GPIO
```

```
# =====
```

```
led = Pin(LED_PIN, Pin.OUT)
```

```
# =====
```

```
# I2C
```

```
# =====
```

```
i2c = I2C(
```

```
    I2C_ID,
```

```
    scl=Pin(SCL_PIN),
```

```
    sda=Pin(SDA_PIN),
```

```
    freq=I2C_FREQ
```

```
)
```

```
# =====
```

```
# SAFE I2C WRITE
```

```
# =====
```

```
def write_register(register, value):
```

```
retry = 3
```

```
while retry > 0:
```

```
    try:
```

```
        i2c.writeto_mem(
            ACC_ADDRESS,
            register,
            bytes([value])
        )
```

```
        return True
```

```
    except OSError:
```

```
        retry -= 1
```

```
        utime.sleep_ms(50)
```

```
    return False
```

```
# =====
```

```
# SAFE I2C READ
```

```
# =====
```

```
def read_registers(register, length):
```

```
    retry = 3
```

```
while retry > 0:
```

```
    try:
```

```
        data = i2c.readfrom_mem(
            ACC_ADDRESS,
            register,
            length
        )
```

```
        return data
```

```
    except OSError:
```

```
        retry -= 1
```

```
        utime.sleep_ms(50)
```

```
    return None
```

```
# =====
```

```
# DEVICE CHECK
```

```
# =====
```

```
def check_sensor():
```

```
    devices = i2c.scan()
```

```
    if ACC_ADDRESS in devices:
```

```
        return True
```

```
return False
```

```
# =====
```

```
# INITIALIZE SENSOR
```

```
# =====
```

```
def initialize_sensor():
```

```
    if not check_sensor():
```

```
        raise Exception("Accelerometer Not Found")
```

```
    write_register(
```

```
        CTRL_REG1_A,
```

```
        0x57
```

```
    )
```

```
    write_register(
```

```
        CTRL_REG4_A,
```

```
        0x08
```

```
    )
```

```
    utime.sleep_ms(100)
```

```
# =====
```

```
# SIGNED CONVERSION
```

```
# =====
```

```
def convert_signed(value):
```

```
value >>= 4
```

```
if value >= 2048:
```

```
    value -= 4096
```

```
return value
```

```
# =====
```

```
# RAW XYZ
```

```
# =====
```

```
def read_accelerometer():
```

```
    data = read_registers(  
        OUT_X_L_A | AUTO_INCREMENT,  
        6  
    )
```

```
    if data is None:
```

```
        return None
```

```
    x = (data[1] << 8) | data[0]
```

```
    y = (data[3] << 8) | data[2]
```

```
    z = (data[5] << 8) | data[4]
```

```
    x = convert_signed(x)
```

```
    y = convert_signed(y)
```

```
    z = convert_signed(z)
```

```
return x, y, z
```

```
# =====
```

```
# CALIBRATION
```

```
# =====
```

```
def calibrate(samples=300):
```

```
    global x_bias
```

```
    global y_bias
```

```
    global z_bias
```

```
    x_sum = 0
```

```
    y_sum = 0
```

```
    z_sum = 0
```

```
    count = 0
```

```
    while count < samples:
```

```
        values = read_accelerometer()
```

```
        if values is None:
```

```
            continue
```

```
        x, y, z = values
```

```
        x_sum += x
```

```
        y_sum += y
```

```
        z_sum += z
```

```

count += 1

    utime.sleep_ms(10)

x_bias = x_sum / samples
y_bias = y_sum / samples
z_bias = (z_sum / samples) - 1024

# =====
# CALIBRATED DATA
# =====

def get_xyz():

    values = read_accelerometer()

    if values is None:

        return None

    x, y, z = values

    x -= x_bias
    y -= y_bias
    z -= z_bias

    return x, y, z

# =====
# ANGLE CALCULATION
# =====

```

```
def calculate_angles(x, y, z):
```

```
    roll = math.atan2(y, z)
```

```
    pitch = math.atan2(
        -x,
        math.sqrt((y * y) + (z * z))
    )
```

```
    roll = math.degrees(roll)
```

```
    pitch = math.degrees(pitch)
```

```
    return roll, pitch
```

```
# =====
```

```
# LOW PASS FILTER
```

```
# =====
```

```
def low_pass_filter(roll, pitch):
```

```
    global filtered_roll
```

```
    global filtered_pitch
```

```
    if filtered_roll is None:
```

```
        filtered_roll = roll
```

```
        filtered_pitch = pitch
```

```
    return filtered_roll, filtered_pitch
```

```
filtered_roll = (  
    ALPHA * roll  
    +  
    (1 - ALPHA) * filtered_roll  
)
```

```
filtered_pitch = (  
    ALPHA * pitch  
    +  
    (1 - ALPHA) * filtered_pitch  
)
```

```
return filtered_roll, filtered_pitch
```

```
# =====  
# MOVEMENT DETECTION  
# =====
```

```
def detect_movement(x, y, z):
```

```
    global previous_x  
    global previous_y  
    global previous_z
```

```
    if previous_x is None:
```

```
        previous_x = x  
        previous_y = y  
        previous_z = z
```

```
    return False
```

```
    dx = abs(x - previous_x)
```

```
    dy = abs(y - previous_y)
```

```
    dz = abs(z - previous_z)
```

```
    previous_x = x
```

```
    previous_y = y
```

```
    previous_z = z
```

```
    if (
```

```
        dx > MOVEMENT_THRESHOLD or
```

```
        dy > MOVEMENT_THRESHOLD or
```

```
        dz > MOVEMENT_THRESHOLD
```

```
    ):
```

```
        return True
```

```
    return False
```

```
# =====
```

```
# TILT DETECTION
```

```
# =====
```

```
def detect_tilt(roll, pitch):
```

```
    if abs(roll) > TILT_THRESHOLD:
```

```
        return True
```

```
    if abs(pitch) > TILT_THRESHOLD:
```

```
    return True
```

```
    return False
```

```
# =====
```

```
# TAMPER DETECTION
```

```
# =====
```

```
def detect_tamper(movement, tilt):
```

```
    if movement and tilt:
```

```
        return True
```

```
    return False
```

```
# =====
```

```
# ALARM
```

```
# =====
```

```
def alarm(status):
```

```
    if status:
```

```
        led.value(1)
```

```
    else:
```

```
        led.value(0)
```

```
# =====
```

```
# DISPLAY
```

```
# =====
```

```
def display_data(
```

```
    x,
```

```
    y,
```

```
    z,
```

```
    roll,
```

```
    pitch,
```

```
    movement,
```

```
    tilt,
```

```
    tamper
```

```
):
```

```
    print("-----")
```

```
    print(
```

```
        "X   :",
```

```
        round(x, 2)
```

```
    )
```

```
    print(
```

```
        "Y   :",
```

```
        round(y, 2)
```

```
    )
```

```
    print(
```

```
        "Z   :",
```

```
        round(z, 2)
```

)

```
print(  
    "Roll :",  
    round(roll, 2)  
)
```

```
print(  
    "Pitch :",  
    round(pitch, 2)  
)
```

```
print(  
    "Move :",  
    movement  
)
```

```
print(  
    "Tilt :",  
    tilt  
)
```

```
print(  
    "Tamper :",  
    tamper  
)
```

```
# =====
```

```
# MAIN PROGRAM
```

```
# =====
```

```
def main():
```

```
print("-----")
print("LSM303DLHC Tamper Detection System")
print("-----")
```

```
initialize_sensor()
```

```
print("Sensor Initialized")
```

```
print("Calibrating...")
```

```
calibrate(300)
```

```
print("Calibration Complete")
```

```
utime.sleep(1)
```

```
while True:
```

```
    try:
```

```
        values = get_xyz()
```

```
        if values is None:
```

```
            continue
```

```
        x, y, z = values
```

```
        roll, pitch = calculate_angles(  
            x,
```

```
            x,
```

```
    y,  
    z  
)
```

```
roll, pitch = low_pass_filter(  
    roll,  
    pitch  
)
```

```
movement = detect_movement(  
    x,  
    y,  
    z  
)
```

```
tilt = detect_tilt(  
    roll,  
    pitch  
)
```

```
tamper = detect_tamper(  
    movement,  
    tilt  
)
```

```
alarm(tamper)
```

```
display_data(  
    x,  
    y,  
    z,
```

```
    roll,  
    pitch,  
    movement,  
    tilt,  
    tamper  
)
```

```
    utime.sleep_ms(100)
```

```
except KeyboardInterrupt:
```

```
    led.value(0)
```

```
    print("Program Stopped")
```

```
    break
```

```
except OSError:
```

```
    print("I2C Communication Error")
```

```
    utime.sleep_ms(200)
```

```
except Exception as e:
```

```
    print("Error :", e)
```

```
    utime.sleep_ms(500)
```

```
# =====
```

```
# START PROGRAM
```

```
# =====
```

```
if __name__ == "__main__":
```

```
    main()
```

explanation

This project is called **Military Equipment Tamper Detection using Raspberry Pi Pico and LSM303DLHC Sensor**. The main purpose of the project is to monitor valuable equipment and detect if it is moved, tilted, lifted, or disturbed without authorization. The system works completely offline and does not require an internet connection, making it suitable for secure environments. The Raspberry Pi Pico acts as the main controller, while the LSM303DLHC sensor provides acceleration data through the I2C communication protocol. First, the Pico initializes the sensor and establishes communication. Then it continuously reads the raw X, Y, and Z acceleration values from the accelerometer. These raw values are converted into signed numbers because the sensor stores data in two's complement format. After that, a calibration process is performed to remove the sensor's bias and improve measurement accuracy. The calibrated values are then used to calculate the roll and pitch angles, which indicate the orientation of the equipment. A low-pass filter is applied to reduce unwanted sensor noise and make the readings stable. The program also compares the current acceleration values with the previous readings to determine whether the equipment has been moved. If the roll or pitch angle exceeds a predefined threshold, the system recognizes that the equipment has been tilted. The movement detection and tilt detection results are combined to make a final tamper decision. If both conditions indicate unauthorized activity, the Raspberry Pi Pico activates an alarm by turning on an LED or a buzzer and displays a tamper alert message in Thonny. If the equipment remains stationary, the alarm stays off and the system continues monitoring. The entire process runs continuously in a loop, allowing real-time monitoring of the equipment. The program also includes basic error handling to recover from temporary I2C communication errors without crashing. This makes the system more reliable for long-term operation.

This project has many practical applications. It can be used to monitor military communication equipment, ammunition storage boxes, portable radar systems, surveillance devices, weapon storage lockers, and field-deployed electronic equipment. It can also be used in defense laboratories, training centers, and secure storage rooms to detect unauthorized handling of sensitive equipment. Beyond defense, the same concept can be applied to bank lockers, museum artifacts, laboratory instruments, industrial machines, server racks, medical equipment, and valuable transport containers. Since the system is compact, low-cost, energy-efficient, and works completely offline, it is suitable for environments where security, reliability, and continuous monitoring are important. The project demonstrates the use of embedded systems, I2C communication, sensor interfacing, signal processing, calibration, filtering, motion analysis, and real-time monitoring, making it an excellent educational and practical project for embedded systems and IoT learning.